



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

Specifica Tecnica_G

Versione	0.0.4
Stato	In corso
Redazione	Giulia Barzon, Joseph Grant, Chiara Grossele
Verifica_G	Verificato
Approvazione	Da approvare
Proprietario	ByteMe
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin

Registro dei Cambiamenti

Versione	Data	Autore	Descrizione	Verifica _G
0.0.4	17/04/2026	Joseph Grant	Aggiunta descrizione delle componenti frontend	
0.0.3	15/04/2026	Chiara Grossele	Modifica struttura e aggiunte alla sezione Architettura backend	Joseph Grant
0.0.2	14/04/2026	Joseph Grant	Aggiunta tecnologie e architettura	
0.0.1	28/03/2026	Giulia Barzon	Creazione del documento, scrittura introduzione	Tommaso Tom- bacco

Tabella 1: Registro delle versioni del documento

Indice

1	Introduzione	1
1.1	Scopo del documento	1
1.2	Approccio alla redazione	1
1.3	Scopo del prodotto	1
2	Glossario_G	1
3	Riferimenti	1
3.1	Riferimenti informativi	2
4	Tecnologie	2
4.1	Framework frontend	2
4.2	Librerie frontend	3
4.3	Linguaggio frontend	3
4.4	Tecnologie di build	3
4.5	Tecnologie di test	4
4.6	Framework api _G backend	4
4.7	Linguaggio backend	4
4.8	Server backend	4
4.9	Tecnologie deployment	5
5	Architettura backend	5
5.1	Presentation Layer	5
5.2	Application Logic	5
5.3	Business Logic	6
5.3.1	Gestione dei Prompt _G (ai_strategies.py)	6
5.3.2	Orchestrazione delle Operazioni (operation_registry.py)	6
5.3.3	Integrazione Modelli LLM (model_strategies.py)	6
5.3.4	Servizi di Supporto al Dominio (text_cleaning.py)	6
6	UML backend	6
7	Deployment	6
8	Architettura frontend	7
8.1	Model	7
8.2	View	7
8.3	ViewModel	7
8.4	TopBar	7
8.5	SideBar	7
8.6	EditorButton	8
8.7	SaveDocumentModal	8
8.8	MarkDownEditor	8
8.9	HistoryList	9
8.10	HistoryCard	9
9	UML frontend	9

1 Introduzione

1.1 Scopo del documento

Il presente documento costituisce la **Specifica Tecnica_G** del prodotto *Second Brain*, proposto dall'azienda *Zucchetti* al gruppo ByteMe.

Il documento descrive in dettaglio:

- le **tecnologie** adottate (linguaggi, framework, librerie, strumenti);
- l'**architettura** logica e fisica del sistema, con i pattern architetturali scelti;
- i **design pattern** applicati a livello di componenti, con motivazione delle scelte;
- la **documentazione API_G** tra frontend e backend;
- i **requisiti soddisfatti** con riferimento al documento *Analisi dei Requisiti_G* v1.4.1.

Il documento è rivolto al team di sviluppo come guida implementativa e a committente e proponente come verifica_G della coerenza tra scelte progettuali e requisiti concordati.

1.2 Approccio alla redazione

Il documento viene redatto in modo **incrementale**, assicurando coerenza con gli sviluppi in corso. Non può essere considerato completo fino al rilascio della versione 1.0.0.

1.3 Scopo del prodotto

Il progetto *Second Brain* mira a sviluppare un editor di testo_G avanzato basato sul linguaggio Markdown_G e potenziato dall'integrazione di Large Language Model (LLM). L'applicazione web consentirà all'utente di:

- scrivere e visualizzare in tempo reale testi formattati in Markdown_G;
- interagire con un LLM per riassumere, correggere, riscrivere e tradurre testi;
- ottenere analisi critiche multi-prospettiche secondo il metodo dei *Sei Cappelli per Pensare* di Edward De Bono;
- delegare all'AI la stesura completa di un testo tramite Distant Writing_G.

2 Glossario_G

Per evitare ambiguità, i termini tecnici usati nel documento vengono definiti in modo più approfondito nel documento Glossario_G v2.0.0 che si può trovare nel sito web del gruppo ByteMe alla sezione Documenti Interni e raggiungibile dal seguente link:

ByteMe/Glossario_G v2.0.0

I termini che vengono considerati meritevoli di approfondimenti sono indicati con una G a pedice.

3 Riferimenti

- *Norme di Progetto_G* versione 2.0.0
ByteMe/Norme di Progetto_G v2.0.0
- Capitolato d'appalto C6: *Second Brain* (Zucchetti S.p.A.)
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>

- Regolamento del progetto didattico:
<https://www.math.unipd.it/~tullio/IS-1/2023/Dispense/PD2.pdf>

3.1 Riferimenti informativi

- *Analisi dei Requisiti_G* versione 2.0.0 (Ultimo accesso: 01/04/2026)
PB - Analisi dei Requisiti_G v2.0.0
- *Piano di Qualifica* versione 1.0.0 (Ultimo accesso: 01/04/2026)
PB - Piano di Qualifica v2.0.0
- Verbali interni ed esterni
- Glossario_G v2.0.0 (Ultimo accesso: 01/04/2026)
Documenti interni - Glossario_G v2.0.0
- Repository_G ufficiale di EasyMDE_G: <https://github.com/Ionaru/easy-markdown-editor>
(Ultimo accesso: 24/03/2026)
- Documentazione linguaggio markdown_G - CommonMark: <https://commonmark.org/>
(Ultimo accesso: 20/02/2026)
- Documentazione React_G 19: <https://react.dev>
Ultimo accesso: 27/03/2026
- Principi del Bulletproof React_G (repository_G ufficiale): <https://github.com/alan2207/bulletproof-react>
(Ultimo accesso: 27/03/2026)
- Documentazione Flask_G: <https://flask.palletsprojects.com>
(Ultimo accesso: 10/03/2026)
- Documentazione Docker_G: <https://docs.docker.com>
(Ultimo accesso: 27/02/2026)
- Documentazione TypeScript: <https://www.typescriptlang.org/docs/>
(Ultimo accesso: 27/03/2026)
- Slide del corso di Ingegneria del Software
(Ultimo accesso: 27/03/2026)
- Principi SOLID (slide prof. Cardin)
- Pattern architetturali (slide prof. Cardin)

4 Tecnologie

4.1 Framework frontend

Nome	versione	Descrizione
React _G	19.2.0	Libreria JavaScript per la costruzione di interfacce utente basate su componenti riutilizzabili.

Tabella 2: Descrizione del framework frontend

4.2 Librerie frontend

Nome	versione	Descrizione
EasyMDE _G	2.20.0	Editor Markdown _G con anteprima in tempo reale, basato su CodeMirror.
Axios	1.7.9	Client HTTP basato su Promise per effettuare richieste REST dal browser e da Node.js.
TanStack Query	-	Libreria per la gestione del fetching, caching e sincronizzazione dello stato server in React _G .
CSS Modules _G	-	Sistema di scope locale per i fogli di stile CSS, che evita conflitti tra classi in componenti diversi.
React _G Hot Toast	-	Libreria leggera per la visualizzazione di notifiche toast personalizzabili in React _G .
eslint	9.31.1	Strumento di linting per JavaScript e TypeScript, che aiuta a mantenere uno stile di codice coerente e a individuare errori.

Tabella 3: Descrizione delle librerie frontend

4.3 Linguaggio frontend

Nome	versione	Descrizione
TypeScript	-	Superset tipizzato di JavaScript che aggiunge tipizzazione statica e migliora la manutenibilità del codice.
TSX	-	Estensione di TypeScript che consente di scrivere markup JSX _G all'interno di file tipizzati, usata per i componenti React _G .

Tabella 4: Descrizione dei linguaggi frontend

4.4 Tecnologie di build

Nome	versione	Descrizione
Vite _G	7.3.1	Build tool moderno e veloce per applicazioni web, con supporto nativo a ES modules e Hot Module Replacement.

Tabella 5: Descrizione delle tecnologie di build

4.5 Tecnologie di test

Nome	versione	Descrizione
Vitest	4.1.4	Framework di testing per applicazioni Vite _G , compatibile con l'API _G di Jest e ottimizzato per TypeScript.
Pytest	7.0.0	Framework di testing per Python, con sintassi semplice e supporto a fixture, parametrizzazione e plugin.
testing library React _G	16.3.2	Libreria per il testing di componenti React _G , che fornisce utilities per interagire con i componenti in modo simile a come un utente reale lo farebbe.

Tabella 6: Descrizione delle tecnologie di test

4.6 Framework api_G backend

Nome	versione	Descrizione
Flask _G	-	Microframework Python per lo sviluppo di API _G REST, leggero ed estensibile tramite plugin.

Tabella 7: Descrizione del framework api_G backend

4.7 Linguaggio backend

Nome	versione	Descrizione
Python	3.12-slim	Linguaggio di programmazione ad alto livello, utilizzato per la logica di business e l'implementazione delle API _G .

Tabella 8: Descrizione del linguaggio backend

4.8 Server backend

Nome	versione	Descrizione
-	-	-

Tabella 9: Descrizione del server backend

4.9 Tecnologie deployment

Nome	versione	Descrizione
Docker _G	-	Piattaforma per la containerizzazione delle applicazioni, che garantisce ambienti riproducibili e portabili.

Tabella 10: Descrizione delle tecnologie di deployment

5 Architettura backend

Il sistema "Second Brain" è stato progettato adottando il pattern architetturale **layered architecture**. L'adozione di questo pattern mira a garantire la rigorosa separazione delle responsabilità e a minimizzare l'accoppiamento tra i moduli software. Il flusso di esecuzione segue un modello strettamente unidirezionale (dall'alto verso il basso): ogni strato forma un'astrazione e gestisce solo la logica pertinente.

Nel nostro sistema gli strati sono stati progettati come strati chiusi: una richiesta si sposta esclusivamente allo strato immediatamente sottostante, ignorando i dettagli implementativi dei livelli superiori o inferiori.

L'architettura logica del sistema si articola nei seguenti strati:

- **Presentation Layer:** Gestione di UI/UX, dati e richieste tramite API_G REST;
- **Application Logic:** Routing e orchestrazione del backend dell'applicazione;
- **Business Logic:** Nucleo dell'applicazione e gestione dei modelli LLM;

5.1 Presentation Layer

È il livello più alto dell'architettura, fisicamente separato dal backend e delegato all'applicativo frontend. Ha il compito di gestire l'interfaccia utente e la user experience, raccoglie gli input testuali e le selezioni dell'utente e invia le richieste al livello sottostante tramite API_G REST. Questo strato non esegue alcuna elaborazione semantica o logica di generazione, delegando l'intero carico computazionale al backend.

5.2 Application Logic

Rappresenta il punto di ingresso logico dell'applicazione lato backend. Nel progetto, questo livello è interamente implementato nel modulo di routing del framework Flask_G all'interno del file app.py.

Questo strato ha diverse responsabilità:

- Agisce come REST Controller e orchestratore del flusso.
- Riceve le richieste HTTP POST provenienti dall'esterno.
- Effettua controlli di validità semplici e sintattici dell'input.
- Recupera la configurazione necessaria interrogando il registro delle operazioni e traduce la richiesta esterna in chiamate alla Business Logic sottostante.
- Gestisce gli errori applicativi, catturando eventuali eccezioni di sistema per costruire e restituire una risposta formattata in un payload JSON standardizzato e sicuro per il client.

L'Application Logic si occupa esclusivamente del routing e dell'orchestrazione: non possiede alcuna conoscenza diretta delle regole di dominio, della struttura dei `promptG` o delle librerie di rete utilizzate per contattare l'Intelligenza Artificiale.

5.3 Business Logic

È il nucleo del sistema, dove risiedono le regole funzionali di Second Brain. Questo strato è rigidamente isolato dall'Application Logic: non possiede alcuna conoscenza del framework web (HTTP, `FlaskG`) né di come i dati verranno presentati o salvati, garantendo così il massimo disaccoppiamento. Al suo interno fa uso di pattern comportamentali (strategy) per gestire la variabilità degli algoritmi in modo pulito ed estensibile.

I componenti principali di questo livello si dividono in servizi di dominio e servizi di integrazione:

- Gestione dei `PromptG`;
- Orchestrazione delle Operazioni;
- Integrazione Modelli LLM;
- Servizi di Supporto al Dominio.

5.3.1 Gestione dei `PromptG` (`ai_strategies.py`)

Incapsula la logica di costruzione dei contesti per l'Intelligenza Artificiale. Tramite il pattern strategy, definisce le regole per le operazioni standard (`SimplePromptStrategy`), per le traduzioni (`TranslationStrategy`) e per l'analisi psicologica del testo con il metodo dei Sei Cappelli di De Bono (`DeBonoHatStrategy`).

5.3.2 Orchestrazione delle Operazioni (`operation_registry.py`)

Implementa il pattern registry. Contiene le strutture dati (`OperationConfig`) che mappano dinamicamente gli identificatori delle richieste alle rispettive classi di `promptG` e di modello. Fornisce all'Application Layer gli oggetti già configurati e pronti all'uso.

5.3.3 Integrazione Modelli LLM (`model_strategies.py`)

Astrae e confina le complessità tecniche di rete per la comunicazione con i provider esterni (infrastruttura Zucchetti, LLM compatibili con standard OpenAI). Si occupa di iniettare le variabili d'ambiente (`APIG keys` e base URL configurati in `.env` e nel `dockerG-compose.yml`) e di lanciare la generazione effettiva. Grazie all'interfaccia `AIModelStrategy`, espone i modelli `Llama 3.2G` e `Gemma 3G` in modo polimorfo, consentendo al sistema di richiedere la generazione del testo senza doversi preoccupare dei formati di richiesta proprietari delle `APIG`.

5.3.4 Servizi di Supporto al Dominio (`text_cleaning.py`)

Un sottomodulo funzionale contenente un algoritmo basato su espressioni regolari (regex) atto a post-processare e "pulire" l'output fornito dai modelli linguistici.

6 UML backend

7 Deployment

Per il deployment abbiamo scelto di usare `DockerG`, che ci permette di creare container isolati per il frontend e il backend, garantendo coerenza tra gli ambienti di sviluppo e produzione. Ogni

componente ha un proprio Dockerfile che definisce le dipendenze e la configurazione necessaria per l'esecuzione.

8 Architettura frontend

Per il frontend, abbiamo adottato un'architettura MVVM (Model-View-ViewModel) che si adatta bene al paradigma dei componenti di React_G. La struttura di file e cartelle che abbiamo individuato prende dalle best practice del Bulletproof React_G, che ci permette di avere una chiara separazione di feature, componenti e hook.

8.1 Model

8.2 View

Componenti dumb che si occupano esclusivamente della presentazione dei dati e dell'interfaccia utente.

8.3 ViewModel

Si occupa della gestione dello stato delle componenti tramite hook realizzati mediante Zustand.

8.4 TopBar

Il componente TopBar contiene dei pulsanti che consentono di formattare il testo secondo la sintassi Markdown_G.

Attributi:

- `documentName:string` - nome del documento

Metodi:

- `onDocumentNameChange:(newName:string) => void` - callback per aggiornare il nome del documento
- `onCloseDocument:() => void` - callback per chiudere il documento

8.5 SideBar

Il componente SideBar contiene i pulsanti con le varie funzionalità AI.

Attributi:

- `documentName:string` - nome del documento

Metodi:

- `activePage:'editor' | 'history'` - tipologia di view visualizzata nel corpo principale della pagina
- `onNavigate:(page:'editor' | 'history') => void` - callback per aggiornare la view visualizzata
- `onUpload:() => void` - callback per caricare un documento
- `onSave:() => void` - callback per salvare un documento

- `onPrint:() => void` - callback per stampare un documento
- `onAiAction:() => void` - callback per attivare un'azione AI

8.6 EditorButton

Il componente `EditorButton` è un bottone che viene utilizzato nella `sideBar` per i bottoni IA.

Attributi:

- `icon:ReactNode` - icona del bottone
- `label:string` - testo del bottone
- `isActive?:boolean` -
- `disabled?:boolean` - true se il bottone non è cliccabile dall'utente

Metodi:

- `onClick:() => void` - callback per gestire il click sul bottone

8.7 SaveDocumentModal

Il componente `SaveDocumentModal` è il componente che rappresenta il modal per il salvataggio del documento.

Attributi:

- `isOpen:boolean` - true se il modal è aperto
- `currentDocumentName:string` - nome attuale del documento, da mostrare come placeholder nell'input del nome del documento
- `isExporting?:boolean` - true se il documento è in fase di esportazione, per disabilitare i bottoni

Metodi:

- `onClose:() => void` - callback per chiudere il modal
- `onExport:(format:ExportFormat, filename:string) => void` - callback per esportare il documento in un formato specifico

8.8 MarkdownEditor

Il componente `MarkdownEditor` è il componente che rappresenta il corpo principale della pagina e dove viene istanziato `EasyMDEG`.

Attributi:

- `textAreaRef: ReactG.RefObject|HTMLTextAreaElement — null>` - riferimento alla textarea su cui istanziare `EasyMDEG`

Metodi:

- `onFileDrop?: (file: File) => void` - callback per gestire il drop di un file sulla textarea

8.9 HistoryList

Il componente HistoryList è il componente che rappresenta la lista delle interazioni con l'IA.

Attributi:

- items: HistoryItem[] - lista delle interazioni con l'IA da mostrare nella cronologia
- isLoading: boolean - true se la cronologia è in fase di caricamento, per mostrare un indicatore di caricamento
- error: string — null - messaggio di errore da mostrare se c'è stato un errore nel caricamento della cronologia

Metodi:

- onDelete: (id: string) => void - callback per gestire la cancellazione di una interazione dalla cronologia

8.10 HistoryCard

Il componente HistoryCard è il componente che rappresenta la scheda di una interazione con l'IA.

Attributi:

- item: HistoryItem

Metodi:

- onDelete: (id: string) => void - callback per gestire la cancellazione di una interazione dalla cronologia

9 UML frontend